

Microsoft Sentinel Teaching Lab — Full Setup & Log Ingestion Guide

Purpose: Build a self-contained Microsoft Sentinel lab for classroom/training use, with synthetic log data pushed in via PowerShell, so students can practice detections without needing real infrastructure.

Audience: Instructor setting up the lab once; students query/investigate afterward.

What you'll end up with:

- An Azure Log Analytics Workspace with Sentinel enabled
- A custom Data Collection Rule (DCR) + custom table for lab data
- PowerShell scripts that simulate 6 common use cases (brute force, malware, network anomaly, insider threat/exfil, phishing, impossible travel)
- Analytics Rules + KQL to detect each scenario
- A real Microsoft Defender XDR trial tenant connected to Sentinel, with an onboarded device, an official Microsoft attack simulation run against it, and genuine Advanced Hunting telemetry (`DeviceProcessEvents`, `AlertInfo`, etc.) flowing into the workspace
- A complete KQL training curriculum, from absolute basics through joins, time series, and full custom-detection design, built entirely against the data you've already ingested
- A full Microsoft Security Copilot for SOC Analysts hands-on curriculum — provisioning, prompt engineering, Defender XDR and Sentinel integration, threat intel enrichment, custom KQL plugins and promptbooks, custom agent development (no-code, YAML, and pro-code MCP paths), and a capstone project — built on the same lab data and incidents
- Verification steps and cleanup instructions

Note on API choice: Microsoft retired the old "HTTP Data Collector API" (Log Analytics Data Collector API) — new workspaces should use the **Azure Monitor Logs Ingestion API** with a Data Collection Rule (DCR). This guide uses that modern path throughout. It requires an extra one-time app registration step, which is included below.

Part 0 — Prerequisites

Requirement	Notes
Azure subscription	Owner or Contributor + User Access Administrator, or ask your Azure admin
PowerShell 7+	<code>Az</code> module and <code>Az.Accounts</code>

Requirement	Notes
Permission to register an Entra ID App	Or ask a tenant admin to do this once
~30-45 min	For first-time full setup

Install the required PowerShell modules once:

```
powershell
```

```
Install-Module -Name Az -Scope CurrentUser -Repository PSGallery -Force
```

```
Install-Module -Name Az.OperationalInsights -Scope CurrentUser -Force
```

```
Import-Module Az
```

```
Connect-AzAccount
```

Part 1 — Create a Resource Group and Log Analytics Workspace (Portal, click-by-click)

1. Go to <https://portal.azure.com> and sign in.
2. In the top search bar, type **Resource groups** → click **Resource groups**.
3. Click + **Create**.
 - **Subscription:** select your subscription.
 - **Resource group name:** `rg-sentinel-lab`
 - **Region:** e.g. `Central India` or `East US` (pick one close to you; use the same region everywhere below).
 - Click **Review + create** → **Create**.
4. In the top search bar, type **Log Analytics workspaces** → click it.
5. Click + **Create**.
 - **Subscription:** your subscription
 - **Resource group:** `rg-sentinel-lab`
 - **Name:** `law-sentinel-lab`
 - **Region:** same as step 3
 - Click **Review + create** → **Create**.
6. Wait for deployment (~1 min) → click **Go to resource**.
7. Note the **Workspace ID** (Overview page) — you'll need it later. Also note the workspace's **Resource ID** (JSON view or Properties blade).

Part 2 — Enable Microsoft Sentinel on the Workspace

1. In the portal search bar, type **Microsoft Sentinel** → click it.
 2. Click + **Create**.
 3. Select the workspace `law-sentinel-lab` from the list.
 4. Click **Add**.
 5. Wait ~30 seconds — Sentinel is now enabled on that workspace.
 6. You'll land on the Sentinel **Overview** dashboard (empty for now — that's expected).
-

Part 3 — Enable a Couple of Built-In Sample Data Connectors (optional but useful)

These give you realistic Microsoft-generated telemetry alongside your synthetic data.

1. In Sentinel, left menu → **Content management** → **Content hub**.
2. Search "Azure Active Directory" (or **Microsoft Entra ID**) → click it → **Install**.
3. Once installed, left menu → **Configuration** → **Data connectors**.
4. Search **Microsoft Entra ID** → click it → **Open connector page** → follow on-screen **Connect** button (needs Entra admin consent — if you're only using synthetic data, you can skip this connector entirely and go straight to Part 4).
5. Repeat for **Azure Activity** connector if you want subscription-level audit logs too.

(If this is a pure teaching sandbox with no real Azure activity, feel free to skip Part 3 and rely entirely on the synthetic data in Part 5.)

Part 4 — Create a Custom Table + Data Collection Rule (DCR) for Lab Data

We'll create **one custom table** called `SentinelLab_CL` that all our synthetic use-case scripts will feed, with a `EventType` column to distinguish scenarios (cleaner than one table per use case, but the guide also shows how to split them if you prefer separate tables per use case).

⚠ **Naming pitfall — read this first.** Azure automatically appends `_CL` to whatever table name you type in the wizard. If you type `SentinelLab_CL` into the **Table name** field, Azure will create `SentinelLab_CL_CL` (the suffix gets doubled). **Type only `SentinelLab`** in the wizard (no `_CL`) and let Azure add the suffix itself, so the final table is `SentinelLab_CL` as intended. After creation, always confirm the real name via Log Analytics → **Tables**, and use that exact name everywhere (KQL queries, `$Cfg.StreamName`, Analytics Rules) — don't assume it matches what you typed.

⚠ **Schema pitfall — also read this first.** DCR-based custom tables only store columns that were declared at creation time (or added later via **Edit schema**). If you ingest a record with a field the table has never seen, Azure Monitor silently **drops that field** rather than erroring — the ingestion call reports success, but the column never appears

and any KQL referencing it fails with *"Failed to resolve scalar expression"*. **To avoid hitting this six separate times across Part 6**, this guide now has you paste a combined sample covering every use case's fields in step 6 below, so the full schema is created upfront.

4.1 Create the custom table (Portal)

1. Go to your Log Analytics workspace `law-sentinel-lab`.
2. Left menu → **Tables**.
3. Click + **Create** → **New custom log (DCR-based)**.
4. **Table name:** `SentinelLab` (Azure appends `_CL` automatically → final name `SentinelLab_CL` — see naming pitfall above; do not type `_CL` yourself).
5. **Data Collection Rule:** click **Create new**
 - **DCR name:** `dcr-sentinel-lab`
 - **Resource group:** `rg-sentinel-lab`
 - Click **Done**.
6. On the **Schema** step, click **Browse** and upload a sample JSON file, or paste sample data directly (click **Edit** if needed to switch to paste mode). Paste this **combined** sample — it covers every field used across all six use cases in Part 6, so no columns get silently dropped later:

json

```
[
  {
    "TimeGenerated": "2026-07-08T10:00:00Z",
    "EventType": "FailedLogin",
    "SourceIP": "203.0.113.45",
    "DestinationIP": "10.0.5.1",
    "DestinationPort": 443,
    "UserName": "jdoe@contoso.com",
    "DeviceName": "WKSTN-042",
    "Application": "0365",
    "Result": "Success",
    "Country": "RU",
    "Protocol": "TCP",
    "Action": "Allowed",
    "BytesSent": 100,
    "Details": "sample",
    "FileName": "invoice_update.exe",
    "FileHash": "e99a18c428cb38d5f260853678922e03",
    "ThreatName": "Trojan:Win32/Emotet",
    "CommandLine": "powershell -enc sample",
    "ParentProcess": "invoice_update.exe",
    "Severity": "High",
    "DataVolumeMB": 145,
    "Destination": "USB-Drive",
    "Recipient": "finance-team@contoso.com",
    "Sender": "ceo-urgent@contoso.co",
    "Subject": "URGENT",
    "AttachmentName": "invoice.pdf.exe",
    "SpamScore": 8.7,
    "Verdict": "Phish",
    "LinkDomain": "contoso-login-verify.com"
  }
]
```

7. Click **Next** to preview the inferred schema. Check every column's inferred type and correct any that are wrong — in particular:

- `TimeGenerated` → `datetime`
- `DestinationPort`, `BytesSent`, `DataVolumeMB` → `int`
- `SpamScore` → `real`
- everything else → `string`

8. Click **Next: Review + Create** → **Create**.

9. Wait ~2-5 minutes for the table and DCR to provision.

10. Once created, go to **Tables** and confirm the exact table name shown (e.g. `SentinelLab_CL`). Use that exact string everywhere from here on.

4.2 Note down 3 values you'll need for PowerShell

1. DCR Immutable ID:

- Portal → search **Data Collection Rules** → click `dcr-sentinel-lab` → **JSON View** (top right) → copy `immutableId`.

2. DCE (Data Collection Endpoint) Log Ingestion URI:

- Same DCR resource → **Overview** → note the **Data collection endpoint** it's linked to → open that DCE resource → copy the **Logs ingestion** URI (looks like `https://dcr-sentinel-lab-xxxx.centralindia-1.ingest.monitor.azure.com`).
- If no DCE was auto-created, portal search **Data Collection Endpoints** → + **Create** → link it to the DCR (Portal → DCR → **Resources/Data sources...** in newer UI this is streamlined; if the wizard auto-created one for you, just use it).

3. **Stream name:** format is always `Custom-<TableName>` using the table's *actual, confirmed* name from step 10 above — e.g. `Custom-SentinelLab_CL`. Verify this directly rather than assuming: Portal → **Data Collection Rules** → `dcr-sentinel-lab` → **JSON view** → look under `properties.dataFlows[0].streams` for the exact string, and use that verbatim.

4.3 Register an Entra ID App (one-time, needed for the Logs Ingestion API)

1. Portal search **App registrations** → + **New registration**.
2. **Name:** `sentinel-lab-ingest-app` → **Register**.
3. Note the **Application (client) ID** and **Directory (tenant) ID** from the Overview page.
4. Left menu → **Certificates & secrets** → + **New client secret** → **Add**. Copy the **Value** immediately (shown once).
5. Now grant this app permission to send logs via the DCR:
 - Portal search **Data Collection Rules** → click `dcr-sentinel-lab`.
 - Left menu → **Access control (IAM)** → + **Add** → **Add role assignment**.
 - Role: **Monitoring Metrics Publisher** → **Next**.
 - **Assign access to:** User, group, or service principal → + **Select members** → search `sentinel-lab-ingest-app` → select it → **Review + assign**.

You now have everything needed for PowerShell ingestion:

- Tenant ID
- Client (App) ID
- Client Secret
- DCE Logs Ingestion URI
- DCR Immutable ID
- Stream name `Custom-SentinelLab_CL`

Part 5 — PowerShell: Reusable Ingestion Function

Save this as `Send-SentinelLabLogs.ps1`. Every use-case script below calls this function.

powershell

```

# Send-SentinelLabLogs.ps1
# Reusable function to push JSON log records into the SentinelLab_CL custom tab
# via the Azure Monitor Logs Ingestion API.

function Get-IngestAccessToken {
    param(
        [string]$TenantId,
        [string]$ClientId,
        [string]$ClientSecret
    )
    $body = @{
        client_id      = $ClientId
        scope           = "https://monitor.azure.com/.default"
        client_secret  = $ClientSecret
        grant_type      = "client_credentials"
    }
    $tokenUri = "https://login.microsoftonline.com/$TenantId/oauth2/v2.0/token"
    $resp = Invoke-RestMethod -Uri $tokenUri -Method Post -Body $body -ContentType application/json
    return $resp.access_token
}

function Send-SentinelLabLogs {
    param(
        [Parameter(Mandatory)] [string]$TenantId,
        [Parameter(Mandatory)] [string]$ClientId,
        [Parameter(Mandatory)] [string]$ClientSecret,
        [Parameter(Mandatory)] [string]$DceUri,           # e.g. https://dcr-sentinel-lab-portal.azure.com
        [Parameter(Mandatory)] [string]$DcrImmutableId,  # e.g. dcr-0123456789abc
        [Parameter(Mandatory)] [string]$StreamName,      # e.g. Custom-SentinelLabLogs
        [Parameter(Mandatory)] [Array]$Records           # array of hashtables/PSObjects
    )

    $token = Get-IngestAccessToken -TenantId $TenantId -ClientId $ClientId -ClientSecret $ClientSecret

    # .Trim() every piece defensively – stray whitespace/hidden characters copied from
    # portal (e.g. a trailing CR from a paste) can silently corrupt the URL and cause a
    # confusing 400 "InvalidRequestPath" error that looks like a totally different problem
    $uri = ($DceUri.Trim().TrimEnd('/')) + "/dataCollectionRules/" + ($DcrImmutableId.Trim()) +
        "/streams/" + ($StreamName.Trim()) + "?api-version=2023-01-01"

    # Uncomment while troubleshooting to see exactly what URL is being called:
    # Write-Host "DEBUG URI: [$uri]" -ForegroundColor Cyan

    $headers = @{
        "Authorization" = "Bearer $token"
        "Content-Type"  = "application/json"
    }

```

```

}

$body = $Records | ConvertTo-Json -Depth 5
# Single-object arrays need to stay arrays for the API
if ($Records.Count -eq 1) { $body = "[$body]" }

try {
    $result = Invoke-RestMethod -Uri $uri -Method Post -Headers $headers -B
    Write-Host "Sent $($Records.Count) record(s) successfully." -ForegroundColor Green
}
catch {
    $status = $_.Exception.Response.StatusCode.value__
    Write-Host "Ingestion failed. STATUS: $status" -ForegroundColor Red
    if ($_.ErrorDetails.Message) {
        Write-Host "BODY: $($_.ErrorDetails.Message)" -ForegroundColor Yellow
    }
    elseif ($_.Exception.Response) {
        $stream = $_.Exception.Response.GetResponseStream()
        $stream.Position = 0
        $reader = New-Object System.IO.StreamReader($stream)
        Write-Host "BODY: $($reader.ReadToEnd())" -ForegroundColor Yellow
    }
}
}

```

Why the changes above matter: the original version's catch block only printed `$_ .Exception.Message`, which for HTTP errors is just the generic "(400) Bad Request" wrapper — it hides the actual JSON error body Azure returns, which is the only thing that tells you *why*. The updated catch block always surfaces that body. The `.Trim()` calls guard against invisible characters (e.g. a stray carriage return picked up from copying a value out of the Azure Portal) silently truncating the URL.

Load it in each session:

```
powershell
```

```
. .\Send-SentinelLabLogs.ps1
```

```
$Cfg = @{
    TenantId      = "<your-tenant-id>"
    ClientId      = "<your-app-client-id>"
    ClientSecret   = "<your-client-secret>"
    DceUri        = "https://dcr-sentinel-lab-xxxx.centralindia-1.ingest.monit
    DcrImmutableId = "dcr-xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx"
    StreamName     = "Custom-SentinelLab_CL"
}
```

⚠ **Don't hardcode the client secret in a script file, especially one you'll share with students.** Prefer an environment variable or an interactive prompt:

```
powershell
```

```
$Cfg.ClientSecret = (Get-Credential -UserName "sentinel-lab-ingest-app" -Mes
```

If a secret is ever pasted into a chat, document, or shared script by mistake, treat it as compromised and rotate it immediately (Portal → **App registrations** → your app → **Certificates & secrets** → delete the old one → create a new one).

✅ **Before running anything in Part 6, verify every value in `$Cfg` against the source of truth in the portal rather than trusting what you typed:**

- `TenantId` / `ClientId` → App registration **Overview** page
- `DceUri` → the DCE resource → **Logs ingestion** URI (no trailing slash)
- `DcrImmutableId` → DCR → **JSON view** → `immutableId`
- `StreamName` → DCR → **JSON view** → `properties.dataFlows[0].streams[0]` (exact string, including case)

A mismatch in any one of these is the single most common cause of ingestion failures in this lab, and each produces a *different, misleading-looking* error (403 for identity/role issues, 400 `InvalidRequestPath` for a malformed URL, 400 with a schema-resolution message for a wrong stream/table name).

Part 6 — Use-Case Sample Data Scripts

Each script generates realistic synthetic events and sends them with the function above. Run these **after** loading `Send-SentinelLabLogs.ps1` and setting `$Cfg`.

6.1 Use Case: Brute-Force / Failed Login Attack

```
powershell
```

```
$attackerIP = "203.0.113.45"
$records = @()
1..15 | ForEach-Object {
    $records += [PSCustomObject]{
        TimeGenerated = (Get-Date).ToUniversalTime().AddSeconds(-($_ * 5)).ToString("o")
        EventType     = "FailedLogin"
        SourceIP      = $attackerIP
        UserName       = "jdoe@contoso.com"
        Application    = "0365"
        Result         = "Failure"
        Country        = "RU"
        Details        = "Invalid password attempt $_"
    }
}
# one successful login right after - classic brute-force-then-success pattern
$records += [PSCustomObject]{
    TimeGenerated = (Get-Date).ToUniversalTime().ToString("o")
    EventType     = "FailedLogin"
    SourceIP      = $attackerIP
    UserName       = "jdoe@contoso.com"
    Application    = "0365"
    Result         = "Success"
    Country        = "RU"
    Details        = "Login succeeded after repeated failures"
}

Send-SentinelLabLogs @Cfg -Records $records
```

6.2 Use Case: Malware Detection Alert

powershell

```
$records = @(
    [PSCustomObject]@{
        TimeGenerated = (Get-Date).ToUniversalTime().ToString("o")
        EventType     = "MalwareDetected"
        DeviceName    = "WKSTN-042"
        SourceIP      = "10.0.5.22"
        UserName      = "asharma"
        FileName      = "invoice_update.exe"
        FileHash      = "e99a18c428cb38d5f260853678922e03"
        ThreatName    = "Trojan:Win32/Emotet"
        Action        = "Quarantined"
        Severity      = "High"
    },
    [PSCustomObject]@{
        TimeGenerated = (Get-Date).ToUniversalTime().AddMinutes(-2).ToString("o")
        EventType     = "ProcessCreated"
        DeviceName    = "WKSTN-042"
        SourceIP      = "10.0.5.22"
        UserName      = "asharma"
        FileName      = "powershell.exe"
        CommandLine   = "powershell -enc SQBFAFgAKABOAGUAdwAtAE8AYgBqAGUAYwB0AC"
        ParentProcess = "invoice_update.exe"
        Severity      = "High"
    }
)
Send-SentinelLabLogs @Cfg -Records $records
```

6.3 Use Case: Network Traffic Anomaly (Port Scan / Beaconing)

powershell

```
$records = @()
$internalHost = "10.0.5.22"
$targetPorts = 1..25 | Get-Random -Count 20
foreach ($port in $targetPorts) {
    $records += [PSCustomObject]@{
        TimeGenerated = (Get-Date).ToUniversalTime().AddSeconds(-(Get-Random -
        EventType     = "NetworkConnection"
        SourceIP      = $internalHost
        DestinationIP = "10.0.5.$(Get-Random -Minimum 1 -Maximum 254)"
        DestinationPort= $port
        Protocol       = "TCP"
        Action         = "Allowed"
        BytesSent      = Get-Random -Minimum 40 -Maximum 200
    }
}
Send-SentinelLabLogs @Cfg -Records $records
```

6.4 Use Case: Insider Threat / Data Exfiltration

powershell

```
$records = @(
    [PSCustomObject]@{
        TimeGenerated = (Get-Date).ToUniversalTime().AddHours(-1).ToString("o")
        EventType     = "FileAccess"
        UserName      = "rpatel"
        DeviceName     = "WKSTN-119"
        FileName       = "Customer_Master_List.xlsx"
        Action         = "Downloaded"
        DataVolumeMB   = 145
        Destination    = "USB-Drive"
    },
    [PSCustomObject]@{
        TimeGenerated = (Get-Date).ToUniversalTime().AddMinutes(-30).ToString("o")
        EventType     = "CloudUpload"
        UserName      = "rpatel"
        DeviceName     = "WKSTN-119"
        FileName       = "Customer_Master_List.xlsx"
        Action         = "UploadedToPersonalCloud"
        Destination    = "personal-dropbox.com"
        DataVolumeMB   = 145
    }
)
Send-SentinelLabLogs @Cfg -Records $records
```

6.5 Use Case: Phishing Email

powershell

```
$records = @(
    [PSCustomObject]@{
        TimeGenerated = (Get-Date).ToUniversalTime().ToString("o")
        EventType     = "EmailReceived"
        Recipient      = "finance-team@contoso.com"
        Sender         = "ceo-urgent@contoso-secure.co"
        Subject        = "URGENT: Wire transfer approval needed today"
        AttachmentName = "Invoice_2026.pdf.exe"
        SpamScore      = 8.7
        Verdict        = "Phish"
        LinkDomain     = "contoso-login-verify.com"
    }
)
Send-SentinelLabLogs @Cfg -Records $records
```

6.6 Use Case: Impossible Travel (Sign-in from two countries in minutes)

powershell

```
$records = @(
    [PSCustomObject]@{
        TimeGenerated = (Get-Date).ToUniversalTime().AddMinutes(-10).ToString("
        EventType     = "SignIn"
        UserName       = "mgarcia@contoso.com"
        SourceIP       = "198.51.100.10"
        Country        = "IN"
        Application    = "0365"
        Result         = "Success"
    },
    [PSCustomObject]@{
        TimeGenerated = (Get-Date).ToUniversalTime().ToString("o")
        EventType     = "SignIn"
        UserName       = "mgarcia@contoso.com"
        SourceIP       = "203.0.113.99"
        Country        = "BR"
        Application    = "0365"
        Result         = "Success"
    }
)
Send-SentinelLabLogs @Cfg -Records $records
```

Tip for a class: wrap all six scripts in a menu-driven `switch` block or a scheduled task so students see data trickle in over the session rather than all at once.

i From this point on, every KQL query below uses `SentinelLab_CL`. That's correct *only if* you typed `SentinelLab` (no `_CL`) into the table-name field in Part 4.1, per the naming pitfall warning. If your actual table came out as something else (e.g. `SentinelLab_CL_CL`), find-and-replace `SentinelLab_CL` with your real table name — confirmed via **Tables** in the Log Analytics workspace — throughout the rest of this document before running any query.

Part 7 — Verify Ingestion (Portal, click-by-click)

1. Go to your Log Analytics workspace `law-sentinel-lab`.
2. Left menu → **Logs**.
3. Close the query wizard pop-up if it appears.
4. In the query editor, paste:

```
kusto
```

```
SentinelLab_CL
```

```
| sort by TimeGenerated desc
```

```
| take 50
```

5. Click **Run**. You should see your synthetic records (may take 2-5 minutes to appear the first time; after that it's near-real-time).

Part 8 — Build Analytics Rules for Each Use Case (Sentinel, click-by-click)

Repeat this pattern per use case, using the matching KQL.

8.1 General steps to create a rule

1. Open **Microsoft Sentinel** → your workspace → left menu → **Configuration** → **Analytics**.
2. Click + **Create** → **Scheduled query rule**.
3. **General tab**: give it a name (e.g. `Lab - Brute Force Login Detected`), severity `Medium`/`High`, tactics (e.g. `Credential Access`) → **Next**.
4. **Set rule logic tab**: paste the KQL query for the use case (below) into the **Rule query** box.
5. Set **Run query every**: `5 minutes`; **Lookup data from the last**: `10 minutes` (adjust as needed).
6. Under **Alert threshold**, set "Is greater than 0".
7. Under **Entity mapping**, map `Account` → `UserName`, `IP` → `SourceIP` (helps investigation graphs).
8. Click **Next: Incident settings** → keep **Create incidents from alerts** = Enabled → **Next**.
9. **Automated response tab**: skip for now (or attach a playbook if you've built one) → **Next**.
10. **Review + create** → **Create**.

8.2 KQL per use case

Brute force:

```
kusto
```

```
SentinelLab_CL
| where EventType == "FailedLogin"
| summarize FailCount = countif(Result == "Failure"),
              SuccessAfter = countif(Result == "Success")
              by SourceIP, Username, bin(TimeGenerated, 10m)
| where FailCount >= 10
```

Malware:

```
kusto

SentinelLab_CL
| where EventType in ("MalwareDetected", "ProcessCreated")
| where Severity == "High"
```

Port scan:

```
kusto

SentinelLab_CL
| where EventType == "NetworkConnection"
| summarize DistinctPorts = dcount(DestinationPort) by SourceIP, bin(TimeGenerated, 1h)
| where DistinctPorts >= 15
```



Insider exfil:

```
kusto

SentinelLab_CL
| where EventType in ("FileAccess", "CloudUpload")
| where Action in ("Downloaded", "UploadedToPersonalCloud")
| summarize TotalMB = sum(DataVolumeMB) by Username, bin(TimeGenerated, 1h)
| where TotalMB > 100
```

Phishing:

```
kusto

SentinelLab_CL
| where EventType == "EmailReceived"
| where Verdict == "Phish" or SpamScore > 7
```

Impossible travel:

```
kusto
```

```
SentinelLab_CL
| where EventType == "SignIn" and Result == "Success"
| summarize Countries = make_set(Country), Times = make_list(TimeGenerated) by
| where array_length(Countries) > 1
```



Part 9 — Investigate an Incident (click-by-click)

1. Sentinel → left menu → **Threat management** → **Incidents**.
 2. Click an incident (e.g. "Lab - Brute Force Login Detected").
 3. Click **Investigate** to open the investigation graph — shows entities (accounts, IPs) and their relationships.
 4. Click on an entity node to see related alerts/activity.
 5. Back on the incident details pane, use **Tasks** to assign checklist items to students; use **Comments** for analyst notes; set **Status** (New → Active → Closed) and **Classification** when resolved.
-

Part 10 — Connect a Real Microsoft Defender XDR Trial Tenant

This part replaces synthetic data with **genuine Defender XDR telemetry** by connecting a trial Microsoft 365 E5 tenant, onboarding a real (or lab VM) Windows device, running Microsoft's own official attack simulation, and streaming that real data into your Sentinel workspace. This gives students the authentic Advanced Hunting schema ((DeviceProcessEvents), (DeviceNetworkEvents), (DeviceFileEvents), (EmailEvents), (IdentityLogonEvents), (AlertInfo), (AlertEvidence), etc.) and Sentinel's built-in Defender analytics rules working exactly as they would in production — nothing simulated at the data layer.

What you'll need:

Requirement	Notes
A Microsoft 365 E5 trial (or standalone Defender for Endpoint Plan 2 trial)	Free 90-day trial, no credit card required for most tenant types
Global Administrator or Security Administrator on that tenant	Needed to onboard devices and consent to the Sentinel connector
One Windows 10/11 device to onboard	A lab VM is fine — doesn't need to be a production machine
The Azure subscription + Sentinel workspace from Parts 0–2	Defender XDR connects into the same workspace

10.1 Sign up for a Microsoft 365 E5 trial (Portal, click-by-click)

1. Go to <https://www.microsoft.com/microsoft-365/enterprise/e5> (or search "Microsoft 365 E5 trial").
2. Click **Try free for 1 month** (actual trial length varies by region/offer — some show 90 days for Defender-specific trials).
3. Sign in with an existing work/school account, or click **Create one** if you need a new tenant. If creating new: enter your business email, verify with the code sent, then fill in your name, company name, and phone number for verification.
4. Choose a **domain name** for the new tenant (e.g. `contosolab123.onmicrosoft.com`) if you don't have a custom domain).
5. Create your admin username and password → **Sign up**.
6. Wait ~2–3 minutes for the tenant to provision → you'll land on the Microsoft 365 admin center.

(Alternative, narrower option: if you only want Defender for Endpoint without the full M365 E5 suite, search for "Microsoft Defender for Endpoint trial" — Microsoft periodically offers a standalone P2 trial through the Microsoft Defender portal itself; see 10.2 below, which will prompt you to start a trial if none is active.)

10.2 Access the Microsoft Defender Portal

1. Go to <https://security.microsoft.com> and sign in with the admin account from 10.1.
2. If prompted, complete the guided setup wizard (region selection, etc.) — defaults are fine for a lab.
3. Left menu → confirm you can see **Assets** → **Devices** (should be empty for now) and **Email & collaboration** (if your license includes Defender for Office 365).

10.3 Onboard a Windows Device (Portal + local script, click-by-click)

1. In the Defender portal, left menu → **Settings** (gear icon, bottom left) → **Endpoints**.
2. Under **Device management**, click **Onboarding**.
3. **Select operating system to start onboarding process:** choose `Windows 10 and Windows 11`.
4. **Deployment method:** choose `Local Script (for up to 10 devices)` for a lab.
5. Click **Download onboarding package** → saves `WindowsDefenderATPOnboardingScript.cmd` (or similar).
6. Copy that script to the Windows device you're onboarding (VM or physical lab machine).
7. On that device, open **Command Prompt as Administrator** and run the script:

```
cmd

WindowsDefenderATPOnboardingPackage.cmd
```

8. The script configures the built-in Microsoft Defender for Endpoint sensor (part of Windows 10/11) to report to your tenant. No separate agent install is needed on modern Windows.
9. Back in the Defender portal, left menu → **Assets** → **Devices**. Wait 5–20 minutes — your device should appear with a green "Active" sensor health status.

10.4 Confirm Telemetry Is Flowing

1. Defender portal → **Assets** → **Devices** → click your onboarded device.
2. Check the **Timeline** tab — you should already see baseline events (logons, process starts) from normal OS activity within a few minutes of onboarding, even before running any simulation.

10.5 Run Microsoft's Official Detection Test / Demonstration Scenarios (Portal, click-by-click)

Update: the standalone "Simulation catalog" / "Tutorials & simulations" menu described in older guides has been retired from the current Defender portal navigation. Microsoft now documents these tests directly on Learn rather than surfacing them as a dedicated portal blade. The steps below reflect the current, working approach.

Microsoft publishes two kinds of safe, sanctioned tests for this purpose — both benign by design, no real malicious payloads involved:

A. EDR detection test — the quickest way to confirm the sensor is reporting and to generate one alert:

1. On the onboarded Windows device, open **Command Prompt as Administrator**.

2. Confirm the device is listening locally: `Test-NetConnection 127.0.0.1 -Port 80` (run in PowerShell first, separately).
3. Back in Command Prompt, run:

```
dos
powershell.exe -NoExit -ExecutionPolicy Bypass -WindowStyle Hidden $ErrorAct
```

This is Microsoft's own documented test command (see learn.microsoft.com/en-us/defender-endpoint/edr-detection) — it targets `127.0.0.1` (localhost) and downloads nothing malicious; the file doesn't exist, so nothing harmful actually runs. Its only purpose is to trip a known EDR detection signature. 4. Within a few minutes, a new alert appears in Defender portal → **Investigation & response** → **Incidents & alerts** → **Alerts**. 5. For Linux or macOS onboarded devices, Microsoft provides equivalent OS-specific DIY test scripts — see the same Learn article for the current download links per platform, since these are periodically rotated.

B. Demonstration scenarios (broader coverage) — for testing specific protection areas beyond a single EDR alert:

1. Go to learn.microsoft.com/en-us/defender-endpoint/defender-endpoint-demonstrations for the current, authoritative list (Microsoft updates this page directly rather than a portal menu).
2. Pick a scenario matching what you want to demonstrate for the class, e.g.:
 - **Attack surface reduction (ASR) rules demonstration** — downloads sample files that trigger each ASR rule.
 - **Controlled folder access (ransomware) demonstration** — downloads and runs a sample file that triggers ransomware-style folder protection.
 - **EDR detection** — the same test as option A above.
3. Each linked page gives the exact download and the steps to run it on the onboarded device — all files provided are explicitly documented by Microsoft as harmless.
4. After running one, check Defender portal → **Incidents & alerts** for the resulting alert, same as option A.

Either path generates real alerts on your onboarded device without needing a special portal feature — pick whichever matches what you want students to see (a single quick alert vs. a specific protection-area demo).

10.6 Connect Microsoft Sentinel to Defender XDR (Portal, click-by-click)

1. Go to **Microsoft Sentinel** → your workspace `law-sentinel-lab` → left menu → **Configuration** → **Data connectors**.
2. Search **Microsoft Defender XDR** (may show as "Microsoft 365 Defender" depending on portal version) → click it → **Open connector page**.

3. Click **Connect** (or **Connect incidents & alerts**, depending on version).
4. If prompted for admin consent, a popup directs you to sign in with your Defender/M365 tenant's Global Admin account and approve the requested permissions (`SecurityEvents.Read.All`) and related Graph scopes) → **Accept**.
5. Toggle on the specific components you want streamed into Sentinel — typically:
 - **Incidents & alerts**
 - **Device events (Advanced Hunting tables)** — this is the toggle that streams the raw `Device*`, `Email*`, `Identity*` tables into your Log Analytics workspace, not just alert summaries.
6. Click **Apply changes**.
7. Status should change to **Connected** within a few minutes.

10.7 Verify Real Data Landed in Your Sentinel Workspace

1. Log Analytics workspace `law-sentinel-lab` → **Logs**.
2. Run:

```
kusto
DeviceProcessEvents
| where Timestamp > ago(1h)
| sort by Timestamp desc
| take 50
```

3. You should see real process-creation events from your onboarded device, including whatever your attack simulation triggered.
4. Also check:

```
kusto
AlertInfo
| where Timestamp > ago(1h)
| sort by Timestamp desc
```

```
kusto
AlertEvidence
| where Timestamp > ago(1h)
| sort by Timestamp desc
```

(If these tables return no rows after 15–20 minutes, double check step 10.6.5 — the raw event tables only populate if "Device events" streaming was explicitly enabled, not just incidents/alerts.)

10.8 Enable Sentinel's Built-In Defender XDR Analytics Rules

Rather than writing detection KQL by hand for these real tables, Sentinel ships pre-built rule templates matched to the Defender XDR schema.

1. Sentinel → **Configuration** → **Analytics** → **Rule templates** tab.
2. Filter or search by data source "**Microsoft Defender XDR**".
3. Pick a relevant template (e.g. "Advanced Multistage Attack Detection" — a Fusion rule that correlates Defender alerts automatically) → click it → **Create rule**.
4. Walk through the wizard (same pattern as Part 8.1) — most Defender XDR templates come pre-filled with working KQL and sensible defaults; adjust thresholds only if needed → **Review + create**.
5. Repeat for a couple more relevant templates (e.g. anything tagged "Initial Access" or "Execution" matching your simulation scenario).

10.9 Trigger and Investigate a Real Incident

1. If you haven't already, re-run the EDR detection test or a demonstration scenario from 10.5 to generate fresh alert activity.
2. Sentinel → **Threat management** → **Incidents** — within a few minutes you should see an incident auto-created either directly from the Defender XDR connector (Defender's own alerts sync in as Sentinel incidents automatically) or from one of the analytics rules enabled in 10.8.
3. Click the incident → **Investigate** to view the real entity graph (device, account, file, IP) built from actual telemetry.
4. Explore the **Alerts** tab on the incident — for Defender-sourced incidents, you'll typically see rich MITRE ATT&CK tagging and process trees pulled directly from the device timeline, which is the main advantage of the real connector over any simulated table.

10.10 Cleanup Notes for This Part

- **Offboard the device** before decommissioning: Defender portal → **Settings** → **Endpoints** → **Offboarding** → select OS → download and run the offboarding script on the device, so it stops reporting.
- **Disconnect the Sentinel connector** if you're tearing down the lab: Sentinel → **Data connectors** → Microsoft Defender XDR → **Disconnect**.
- The Microsoft 365 E5 / Defender trial itself will expire automatically at the end of its trial period; if you want to end it early, go to the **Microsoft 365 admin center** → **Billing** → **Your products** → find the trial → **Cancel subscription**.
- Deleting the `rg-sentinel-lab` resource group (Part 12) removes the Sentinel workspace and Azure-side resources, but does **not** touch the M365/Defender tenant itself — that's managed separately and needs the steps above.

Part 11 — KQL Complete Training: From Scratch to Advanced

This is a self-contained KQL (Kusto Query Language) curriculum built entirely around the data you already have in this lab — `SentinelLab_CL` from Part 6, and if you completed Part 10, the real Defender XDR tables (`DeviceProcessEvents`, `DeviceNetworkEvents`, `AlertInfo`, etc.). Work through it top to bottom; each module builds on the last. Run every query in **Log Analytics → Logs** (or **Sentinel → Logs**, or **Advanced hunting** in the Defender portal — the language is identical everywhere).

i If a query references `SentinelLab_CL` and your table came out named differently (see Appendix C.4), substitute your real table name throughout.

11.1 Module 1 — Absolute Basics

The simplest possible query — just the table name:

```
kusto  
  
SentinelLab_CL
```

Returns everything (capped at the portal's default row limit). Every KQL query starts with a table (or a `let` statement referencing one).

take — limit rows, no particular order (fast, good for exploring):

```
kusto  
  
SentinelLab_CL  
| take 10
```

sort by / **order by** — these are synonyms:

```
kusto  
  
SentinelLab_CL  
| sort by TimeGenerated desc  
| take 10
```

project — pick specific columns (like SQL's `SELECT`):

```
kusto
```

```
SentinelLab_CL
| project TimeGenerated, EventType, UserName, SourceIP
| take 10
```

project-away — keep everything *except* named columns:

```
kusto

SentinelLab_CL
| project-away TenantId, _ResourceId, Type
| take 10
```

Exercise 1.1: Show the 5 most recent **FailedLogin** events with only **TimeGenerated**, **UserName**, and **SourceIP**.

<details><summary>Solution</summary>

```
kusto

SentinelLab_CL
| where EventType == "FailedLogin"
| project TimeGenerated, UserName, SourceIP
| sort by TimeGenerated desc
| take 5
```

</details>

11.2 Module 2 — Filtering

where — the core filter operator:

```
kusto

SentinelLab_CL
| where EventType == "FailedLogin"
```

String comparisons (KQL string operators, roughly fastest → most flexible):

Operator	Meaning	Example
<code>==</code>	exact match, case-sensitive	<code>Result == "Failure"</code>
<code>=~</code>	exact match, case-insensitive	<code>Result =~ "failure"</code>
<code>has</code>	whole-word match, case-insensitive, indexed (fast)	<code>Details has "password"</code>
<code>contains</code>	substring match, case-insensitive, not indexed (slower)	<code>Details contains "pass"</code>
<code>startswith</code> / <code>endswith</code>	prefix/suffix match	<code>SourceIP startswith "203."</code>
<code>in (...)</code>	value in a list	<code>Country in ("RU", "CN", "KP")</code>

Prefer `has` over `contains` when matching whole words — `has` uses the term index and is significantly faster at scale; `contains` forces a full scan.

Combining conditions:

```
kusto
SentinelLab_CL
| where EventType == "FailedLogin" and Result == "Failure"
| where Country in ("RU", "CN", "KP")
```

Time filtering — always filter time first when possible (KQL scans time-partitioned data, so a `where TimeGenerated > ago(...)` near the top of a query is far more efficient than at the end):

```
kusto
SentinelLab_CL
| where TimeGenerated > ago(24h)
| where EventType == "FailedLogin"
```

Common time literals: `ago(1h)`, `ago(7d)`, `ago(30m)`, `now()`, or an explicit range with `between`:

```
kusto
SentinelLab_CL
| where TimeGenerated between (datetime(2026-07-01) .. datetime(2026-07-09))
```

Exercise 2.1: Find all events in the last 6 hours where the country is Russia ((RU)) and the result was a success.

<details><summary>Solution</summary>

```
kusto

SentinelLab_CL
| where TimeGenerated > ago(6h)
| where Country == "RU" and Result == "Success"
```

</details>

11.3 Module 3 — Aggregation with (summarize)

This is the single most important operator in KQL for security work — almost every detection rule in Part 8 uses it.

Basic count:

```
kusto

SentinelLab_CL
| where EventType == "FailedLogin"
| summarize count()
```

Count grouped by a column ((by)):

```
kusto

SentinelLab_CL
| where EventType == "FailedLogin"
| summarize FailureCount = count() by SourceIP
| sort by FailureCount desc
```

Common aggregation functions:

Function	Purpose
<code>count()</code>	row count
<code>dcount(col)</code>	approximate distinct count (fast, slightly probabilistic)
<code>countif(cond)</code>	conditional count
<code>sum(col)</code> , <code>avg(col)</code> , <code>min(col)</code> , <code>max(col)</code>	numeric aggregates
<code>make_set(col)</code>	array of distinct values
<code>make_list(col)</code>	array of all values (with duplicates)
<code>arg_max(col, *)</code> / <code>arg_min(col, *)</code>	return the whole row with the max/min value of <code>col</code>

`bin()` — group into time buckets (essential for rate-based detections):

```
kusto
SentinelLab_CL
| where EventType == "FailedLogin"
| summarize FailCount = countif(Result == "Failure") by SourceIP, bin(TimeGenerated, 10)
| where FailCount >= 10
```

This is exactly the brute-force detection query from Part 8 — now you know how it works.

`arg_max` — a very common "give me the latest record per group" pattern:

```
kusto
SentinelLab_CL
| where EventType == "SignIn"
| summarize arg_max(TimeGenerated, *) by Username
```

Returns one row per `Username` — whichever row has the highest `TimeGenerated`.

Exercise 3.1: For every `Username`, show the distinct countries they signed in from, and how many total sign-in events they had.

<details><summary>Solution</summary>

```
kusto
```

```
SentinelLab_CL
| where EventType == "SignIn"
| summarize Countries = make_set(Country), TotalSignIns = count() by UserName
```

</details>

Exercise 3.2 (harder): Find any user who signed in successfully from more than one distinct country — the "impossible travel" pattern from Part 8, but built step by step.

<details><summary>Solution</summary>

```
kusto

SentinelLab_CL
| where EventType == "SignIn" and Result == "Success"
| summarize DistinctCountries = dcount(Country), Countries = make_set(Country)
| where DistinctCountries > 1
```

</details>

11.4 Module 4 — Shaping Data: `extend`, string functions, `case`

`extend` — add a computed column without replacing existing ones:

```
kusto

SentinelLab_CL
| where EventType == "MalwareDetected"
| extend IsHighSeverity = (Severity == "High")
```

Useful string functions:

Function	Purpose	Example
<code>strlen(s)</code>	string length	<code>strlen(FileName)</code>
<code>tolower(s)/ toupper(s)</code>	case conversion	<code>tolower(Username)</code>
<code>strcat(a, b, ...)</code>	concatenate	<code>strcat(Username, "@", "contoso.com")</code>
<code>split(s, delim)</code>	split into a dynamic array	<code>split(FileName, ".")</code>
<code>substring(s, start, len)</code>	substring	<code>substring(SourceIP, 0, 3)</code>
<code>replace_string(s, old, new)</code>	find/replace	<code>replace_string(Details, "Invalid", "Bad")</code>
<code>parse_json(s)</code>	parse a JSON string into <code>dynamic</code>	see Module 6

`case()` — inline if/elif/else logic:

```
kusto
SentinelLab_CL
| where EventType == "MalwareDetected"
| extend RiskLabel = case(
    Severity == "High", "🔴 Critical",
    Severity == "Medium", "🟡 Review",
    "🟢 Low"
)
```

`iff()` — simple two-way conditional (like a ternary):

```
kusto
SentinelLab_CL
| extend IsFailure = iff(Result == "Failure", 1, 0)
```

Exercise 4.1: Extract just the file extension from every `FileName` in malware events, and count occurrences per extension.

<details><summary>Solution</summary>

```
kusto
```

```
SentinelLab_CL
| where isnotempty(FileName)
| extend Extension = toString(split(FileName, ".")[-1])
| summarize count() by Extension
```

</details>

11.5 Module 5 — **let** Statements and Reusable Logic

let defines a named value, table, or even a reusable function scoped to the query.

Simple scalar **let**:

```
kusto

let LookbackWindow = 24h;
let FailureThreshold = 10;
SentinelLab_CL
| where TimeGenerated > ago(LookbackWindow)
| where EventType == "FailedLogin"
| summarize FailCount = count() by SourceIP
| where FailCount >= FailureThreshold
```

This is the pattern used throughout Part 8's analytics rules, and it's good practice generally — tunable thresholds at the top, logic below, nothing hardcoded inline.

let as a saved sub-query (a named table expression):

```
kusto

let SuspiciousIPs = SentinelLab_CL
    | where EventType == "FailedLogin"
    | summarize FailCount = count() by SourceIP
    | where FailCount >= 10
    | project SourceIP;
SentinelLab_CL
| where SourceIP in (SuspiciousIPs)
| where EventType != "FailedLogin"
```

This finds *other* activity (not the failed logins themselves) coming from IPs that already tripped the brute-force threshold — a classic pivot-and-correlate pattern.

let as a reusable function:

```
kusto
```

```
let GetHighSeverityEvents = (minSeverity:string) {
    Sentinellab_CL
    | where EventType == "MalwareDetected"
    | where Severity == minSeverity
};
GetHighSeverityEvents("High")
```

Exercise 5.1: Using `let`, define a 1-hour lookback window and a 100 MB threshold, then find any user whose combined `FileAccess`/`CloudUpload` volume in that window exceeds the threshold (the insider-threat pattern from Part 8, parameterized).

<details><summary>Solution</summary>

```
kusto

let Window = 1h;
let Threshold = 100;
Sentinellab_CL
| where TimeGenerated > ago(Window)
| where EventType in ("FileAccess", "CloudUpload")
| summarize TotalMB = sum(DataVolumeMB) by UserName
| where TotalMB > Threshold
```

</details>

11.6 Module 6 — Joins and Correlation

Real investigations almost always require correlating across event types — this is where `join` and `union` come in.

`union` — stack rows from multiple tables/queries vertically:

```
kusto

union Sentinellab_CL, DeviceProcessEvents
| where TimeGenerated > ago(1h)
| project TimeGenerated, EventType = coalesce(EventType, ActionType)
```

(Only run the `DeviceProcessEvents` half if you completed Part 10 — otherwise just `union` isn't very interesting with one table, but the syntax is worth knowing.)

`join` — combine rows from two tables based on a matching key, side by side:

```
kusto
```

```

let FailedLogins = SentinelLab_CL
  | where EventType == "FailedLogin" and Result == "Failure";
let SuccessfulLogins = SentinelLab_CL
  | where EventType == "FailedLogin" and Result == "Success";
FailedLogins
| join kind=inner (SuccessfulLogins) on SourceIP, UserName
| project SourceIP, UserName, FailTime = TimeGenerated, SuccessTime = TimeGener

```

Note the automatic `(1)` suffix (`(TimeGenerated1)`) — when both sides of a join have a column with the same name, KQL disambiguates the right-hand side by appending `(1)`.

Join kinds you'll actually use:

Kind	Behavior
<code>inner</code> (default)	only rows with matches on both sides
<code>leftouter</code>	all left rows, matched right columns or nulls
<code>leftsemi</code>	left rows that <i>have</i> a match (right columns dropped) — good for "does X exist elsewhere" checks
<code>leftanti</code>	left rows that <i>have no match</i> — great for "find failed logins with NO successful login after them"

`leftanti` example — find brute-force attempts that never succeeded (arguably more suspicious than ones that did):

```

kusto

let AllFailures = SentinelLab_CL | where EventType == "FailedLogin" and Result
let AllSuccesses = SentinelLab_CL | where EventType == "FailedLogin" and Result
AllFailures
| join kind=leftanti (AllSuccesses) on SourceIP, UserName
| summarize FailCount = count() by SourceIP, UserName

```

Exercise 6.1 (only if Part 10 completed): Join your simulated identity logon activity conceptually against real `(DeviceLogonEvents)`/`(DeviceProcessEvents)` by `(DeviceName)` to see what processes ran around the time of a logon. (Open-ended — the goal is practicing the join pattern against real Defender tables.)

11.7 Module 7 — Working with `dynamic` (JSON) Data

Several real Defender XDR tables (and any JSON-shaped custom field you might ingest) use the `dynamic` type.

`parse_json()` — turn a JSON string column into a queryable object:

```
kusto

print RawJson = '{"user":"jdoe","roles":["admin","reader"]}'
| extend Parsed = parse_json(RawJson)
| extend UserName = tostring(Parsed.user)
| extend FirstRole = tostring(Parsed.roles[0])
```

`mv-expand` — explode an array column into multiple rows:

```
kusto

print RawJson = '{"user":"jdoe","roles":["admin","reader"]}'
| extend Parsed = parse_json(RawJson)
| mv-expand Role = Parsed.roles
| project UserName = tostring(Parsed.user), Role = tostring(Role)
```

This turns one row with a 2-element array into two rows, one per role — essential when you need to filter or aggregate on individual array elements.

`bag_unpack` — turn a JSON object's keys into columns:

```
kusto

print RawJson = '{"user":"jdoe","dept":"finance"}'
| extend Parsed = parse_json(RawJson)
| evaluate bag_unpack(Parsed)
```

11.8 Module 8 — Time-Series Analysis and Visualization

`render` — visualize directly in the Logs query results pane:

```
kusto

SentinelLab_CL
| where EventType == "FailedLogin"
| summarize FailCount = count() by bin(TimeGenerated, 1h)
| render timechart
```

Other useful render types: `barchart`, `piechart`, `columnchart`, `scatterchart`.

make-series — build a proper gap-filled time series (unlike **summarize ... by bin()**, this fills in zero-count buckets rather than omitting them):

```
kusto

SentinelLab_CL
| where EventType == "FailedLogin"
| make-series FailCount = count() default=0 on TimeGenerated from ago(24h) to now()
| render timechart
```

series_decompose_anomalies() — statistical anomaly detection over a time series (genuinely advanced — this is the same class of function behind several built-in Sentinel/Fusion detections):

```
kusto

SentinelLab_CL
| where EventType == "FailedLogin"
| make-series FailCount = count() default=0 on TimeGenerated from ago(7d) to now()
| extend (Anomalies, Score, Baseline) = series_decompose_anomalies(FailCount, 1)
| mv-expand TimeGenerated, FailCount, Anomalies, Score, Baseline
| where Anomalies != 0
```

This flags time buckets where failed-login volume deviates significantly from the learned baseline — no manual threshold needed. Real-world caveat: it needs enough historical data (ideally several days to weeks) to establish a meaningful baseline; on a fresh lab table with a few hours of synthetic data, it may not produce meaningful results — worth demonstrating the syntax even if the lab data is too sparse for a realistic result.

11.9 Module 9 — Advanced Analytics Functions

These are genuinely advanced, worth a mention even if lab data volume is too small to show interesting results — students will meet them in production tenants.

autocluster — automatically groups similar rows to surface patterns (huge time-saver in triage: turns 10,000 similar alerts into a handful of clusters):

```
kusto

SentinelLab_CL
| where EventType == "FailedLogin"
| project UserName, SourceIP, Country, Application
| evaluate autocluster()
```

basket — market-basket-style analysis, finds frequently co-occurring combinations of values:

```
kusto

SentinelLab_CL
| where EventType == "MalwareDetected"
| project FileName, ThreatName, Severity
| evaluate basket()
```

****bag_unpack** + **pivot**-style reshaping — covered in 11.7.

Externaldata — query data that lives outside Log Analytics (e.g. a public threat-intel CSV) and join it in:

```
kusto

let KnownBadIPs = externaldata(BadIP: string)
    [@"https://raw.githubusercontent.com/example/threat-feed/main/ips.csv"]
    with (format="csv", ignoreFirstRecord=true);
SentinelLab_CL
| where SourceIP in (KnownBadIPs)
```

(Only works with publicly reachable URLs the workspace can fetch — a good concept to know even if you don't have a live feed handy in the lab.)

11.10 Module 10 — Building a Full Detection From Scratch (Capstone)

Put it all together. Walk through designing a detection the way an analyst actually would, rather than starting from a finished query.

Scenario: detect potential credential-stuffing — many *different* usernames failing login from the *same* source IP in a short window (distinct from brute force, which is many *failures* against *one* username).

Step 1 — start broad, look at the shape of the data:

```
kusto

SentinelLab_CL
| where EventType == "FailedLogin" and Result == "Failure"
| summarize count() by SourceIP
| sort by count_ desc
```

Step 2 — add the "many distinct usernames" dimension:

```
kusto
```

```
SentinelLab_CL
| where EventType == "FailedLogin" and Result == "Failure"
| summarize DistinctUsers = dcount(UserName), TotalAttempts = count() by Source
| sort by DistinctUsers desc
```

Step 3 — add a time window so it's a rate, not an all-time total:

```
kusto

SentinelLab_CL
| where EventType == "FailedLogin" and Result == "Failure"
| summarize DistinctUsers = dcount(UserName), TotalAttempts = count() by Source
| sort by DistinctUsers desc
```

Step 4 — apply a threshold and parameterize it with `let`:

```
kusto

let Window = 15m;
let UserThreshold = 5;
SentinelLab_CL
| where EventType == "FailedLogin" and Result == "Failure"
| summarize DistinctUsers = dcount(UserName),
              TotalAttempts = count(),
              UsersSampled = make_set(UserName)
              by SourceIP, bin(TimeGenerated, Window)
| where DistinctUsers >= UserThreshold
```

Step 5 — this is now ready to become a Sentinel Analytics Rule. Follow the same click-by-click process as Part 8.1, pasting this query into the **Set rule logic** tab, and map `IP` → `SourceIP` in entity mapping.

Capstone exercise for students: design and build, from scratch using the same 5-step process, a detection for "a single user account performing an unusually high volume of `CloudUpload` events within a short window" (a variant insider-threat pattern distinct from the total-MB version in Part 8). Have them present their query and explain each step's purpose — this is a good graded/reviewed lab exercise.

11.11 Quick Reference — Operators Covered

Category	Operators/Functions
Row selection	<code>where</code> , <code>take</code> , <code>top</code> , <code>sort by</code> / <code>order by</code>

Category	Operators/Functions
Column selection	<code>project</code> , <code>project-away</code> , <code>project-rename</code> , <code>extend</code>
Aggregation	<code>summarize</code> , <code>count()</code> , <code>dcount()</code> , <code>countif()</code> , <code>sum()</code> , <code>avg()</code> , <code>min()</code> , <code>max()</code> , <code>make_set()</code> , <code>make_list()</code> , <code>arg_max()</code> , <code>arg_min()</code>
Time	<code>ago()</code> , <code>now()</code> , <code>bin()</code> , <code>between</code> , <code>make-series</code>
Strings	<code>has</code> , <code>contains</code> , <code>startswith</code> , <code>endswith</code> , <code>strcat</code> , <code>split</code> , <code>substring</code> , <code>tolower</code> , <code>toupper</code> , <code>replace_string</code>
Conditionals	<code>case()</code> , <code>iff()</code>
Variables/functions	<code>let</code>
Combining tables	<code>union</code> , <code>join</code> (<code>inner</code> , <code>leftouter</code> , <code>leftsemi</code> , <code>leftanti</code>)
JSON/dynamic	<code>parse_json</code> , <code>mv-expand</code> , <code>bag_unpack</code>
Visualization	<code>render timechart</code> / <code>barchart</code> / <code>piechart</code>
Advanced analytics	<code>series_decompose_anomalies</code> , <code>evaluate autocluster()</code> , <code>evaluate basket()</code> , <code>externaldata</code>

Part 12 — Microsoft Security Copilot for SOC Analysts (Hands-On Labs)

This part maps directly to a standard "Security Copilot for SOC Analysts" training outline, with a click-by-click hands-on lab for every module. It reuses data and infrastructure you already built: `SentinelLab_CL` (Part 6), the real Defender XDR tenant and simulated alerts (Part 10), and the KQL skills from Part 11.


Licensing note, read first: Security Copilot is billed separately from everything else in this guide, using Security Compute Units (SCUs) — either included automatically with Microsoft 365 E5/E7 (400 SCUs/month per 1,000 licenses, up to 10,000/month) or purchased standalone (minimum 1 SCU, ~\$4/hour list price at time of writing). If you completed Part 10's M365 E5 trial, check whether Security Copilot inclusion has activated for your tenant before provisioning standalone capacity — provisioning both is wasted spend. Pricing and inclusion terms change; verify current figures at microsoft.com/en-us/security/pricing/microsoft-security-copilot before a live class.

12.1 Introduction to Security Copilot and SOC Modernization

Security Operations Challenges (discussion, no lab): Frame the module with the problem Security Copilot addresses — alert fatigue, fragmented tools, the KQL skills gap, and slow

manual triage. Use your own Part 6-10 experience as a live example: how many manual steps did it take to hand-build one detection rule and investigate one incident? That's the baseline Copilot is meant to compress.

Introduction to Security Copilot: A generative AI assistant, available as a standalone web app (securitycopilot.microsoft.com) and embedded directly inside Defender, Sentinel, Entra, Intune, and Purview. It answers natural-language questions grounded in your actual tenant data via **plugins**, and can execute multi-step workflows via **promptbooks** and autonomous **agents**.

Security Copilot Architecture: Explain the four building blocks students will use throughout this part:

Component	What it is
Security Compute Units (SCUs)	The billed compute capacity powering every prompt
Plugins	Connectors that give Copilot access to a data source (Defender XDR, Sentinel, Threat Intelligence, third-party)
Promptbooks	Saved, multi-step prompt sequences for a repeatable workflow (e.g. "Incident investigation")
Agents	Autonomous, multi-step AI workflows that act with defined guardrails (e.g. phishing triage agent)

Hands-on Lab 12.1 — Provision Security Copilot (Portal, click-by-click)

1. Confirm your license path first: Microsoft 365 admin center → **Billing** → **Your products** → look for **Microsoft 365 E5 Security** or **E7**. If present, Security Copilot may already be auto-provisioned — skip to step 6 and just sign in.
2. If not auto-provisioned, go to <https://securitycopilot.microsoft.com> and select **Get started**.
3. **Set up a workspace** — enter a workspace name (tenant-bound, cannot be changed later) → **Continue**.
4. Select your **Azure subscription** and **resource group** (reuse `rg-sentinel-lab` from Part 1) where SCU capacity will be provisioned.
5. Set **Security Compute Units**: enter `3` (Microsoft's recommended minimum for an introductory exploration), and set overage to a small bounded number (e.g. `2`) rather than unlimited, to control lab costs → review the data-sharing options → accept terms → **Review + create**.
6. Wait a few minutes for the workspace to deploy, then you'll land on the Security Copilot home page.

7. Confirm your role: **Settings** (gear icon) → **Roles** — you should see yourself listed as **Owner** if you provisioned the capacity (inherited automatically from Security/Global Administrator).
8. Explore the home page: note the **agents-first landing** (if your tenant has this rollout) versus **All history** → **New session** for classic chat.

Discussion checkpoint: Ask students to identify, from the roles list, who else in the class's fictional organization should hold Owner vs. Contributor access, and why (ties into 12.9's governance module later).

12.2 Prompt Engineering for SOC Analysts

Security Prompting Fundamentals: A good security prompt has four parts — **goal** (what you want), **context** (specific entity, time range, incident ID), **source** (which plugin/data should it use), and **format** (table, summary, report). Vague prompts ("check for threats") burn SCUs on unfocused, low-value responses.

Effective Prompt Techniques:

- Be specific with identifiers: `incident ID 15324` beats "the recent incident."
- Anchor time ranges explicitly: "in the last 7 days" beats "recently."
- Iterate: treat the first response as a draft — follow up with "narrow this to only devices tagged production" rather than starting over.
- Use suggested prompts: Copilot surfaces context-aware follow-up prompts on incident/entity pages — these are pre-validated against the actual schema and a good way to learn effective phrasing.

SOC Investigation Prompts (sample library for the class to reuse):

```
Summarize incident <ID> and list all affected devices and users.  
What is the reputation of IP <address>?  
Show me all sign-ins from <user> in the last 24 hours.  
Generate a KQL query for all PowerShell processes with encoded commands in  
the last 7 days.  
What are the recommended next steps for incident <ID>?  
Show me threat intelligence for <threat actor name>.
```

Hands-on Lab 12.2 — Practice Prompting (Portal, click-by-click)

1. Go to securitycopilot.microsoft.com → **All history** → **New session**.
2. Try a vague prompt first: type `check for threats` and send it. Note how generic/unfocused the response is.
3. Now try a well-formed prompt: `Summarize security activity for the last 24 hours across connected sources, grouped by severity.`

4. Compare the two responses as a class — discuss what made the second one more useful.
 5. Select **See all system capabilities** (Prompts icon in the prompt bar) to browse the built-in prompt catalog for whichever plugins are currently enabled.
-

12.3 Security Copilot with Microsoft Defender XDR

Defender XDR Overview: Recap Part 10 — your trial tenant, onboarded device, and the alert(s) generated from the official detection test/demonstration scenario.

Copilot Integration with Defender: Copilot is embedded directly in the Defender portal via a side panel, opened from the **Copilot** button in the top navigation bar. It auto-generates incident summaries, offers **Recommendations** for next steps, can generate a full **Report**, and includes a **query assistant** that turns natural language into ready-to-run KQL for advanced hunting.

Endpoint Investigation: Copilot can summarize a specific device's activity, list related alerts, and answer questions scoped to that device — accessible from the device's own page in **Assets → Devices**.

Hands-on Lab 12.3 — Investigate Your Real Incident with Copilot (Portal, click-by-click)

1. Go to **security.microsoft.com** → **Incidents & alerts** → **Incidents**.
 2. Open the incident generated by your Part 10.5 detection test/demonstration scenario.
 3. The **Copilot pane** opens automatically on the right and begins generating an incident summary — wait for it to complete (or select **Cancel/Regenerate** to control this).
 4. Review the summary, then select **Recommendations** to see AI-suggested next investigative/remediation steps.
 5. Select **Report** to generate a full incident report (summary, timeline, entities) — this is the artifact you'll reuse in Part 12.8.
 6. Select **See prompts** to view suggested follow-up prompts tailored to this incident's entities — pick one (e.g. a device or account summary) and run it.
 7. Go to **Assets → Devices** → your onboarded device → open the **Copilot** panel here too, and ask: `Summarize all activity on this device in the last 24 hours.`
 8. Navigate to **Advanced hunting**. In the Copilot side pane, ensure **Query assistant** mode is selected (toggle from **Threat Hunting Agent** if needed). Ask: `Show me all process creation events involving powershell.exe with an encoded command in the last 7 days.` Review the generated KQL, select **See the logic behind the query**, then **Run query**.
-

12.4 Security Copilot with Microsoft Sentinel

Sentinel Integration: Sentinel connects to Security Copilot in two ways: (1) through the

standalone portal via two dedicated plugins, and (2) natively inside the unified Defender portal's advanced hunting if your Sentinel workspace is connected to Defender XDR.

KQL Generation Using Copilot: The Natural language to KQL for Microsoft Sentinel (Preview) plugin turns questions into hunting queries against your actual Sentinel tables — including custom tables like `SentinelLab_CL`, once configured as context.

Threat Hunting: The built-in **Microsoft Sentinel incident investigation** promptbook is a ready-made starting point — it produces a report covering a specific incident plus related alerts, reputation scores, users, and devices.

Hands-on Lab 12.4 — Connect Sentinel and Generate KQL from Natural Language

1. Go to securitycopilot.microsoft.com → open **Sources** in the prompt bar.
2. Under **Plugins**, turn on **Microsoft Sentinel (Preview)** and **Natural language to KQL for Microsoft Sentinel (Preview)**.
3. Select the gear icon on the **Microsoft Sentinel (Preview)** plugin → set the **default workspace** to `law-sentinel-lab` → **Save**.
4. Start a new session and prompt: `What are the top 5 high priority Sentinel incidents in workspace "law-sentinel-lab"?`
5. Now prompt for KQL directly: `Generate a KQL query against SentinelLab_CL that finds any SourceIP with more than 10 failed logins in a 10-minute window.`
Compare the generated query against the hand-written one you built in Part 8 — discuss any differences in approach.
6. Run the **Microsoft Sentinel incident investigation** promptbook (search for it in the promptbook catalog) against the brute-force incident you created in Part 8/9 — review the generated report.
7. Back in the Defender portal's **Advanced hunting**, confirm you can also query Sentinel tables from the unified query assistant (note: not all Sentinel tables are supported yet — custom tables' support varies, so if `SentinelLab_CL` isn't recognized here, fall back to the standalone portal from step 4).

12.5 Threat Intelligence and Malware Analysis

Threat Intelligence Operations: The **Microsoft Threat Intelligence** plugin gives Copilot access to Defender TI's threat actor profiles, IOCs, tools, and vulnerability data — usable from both the standalone portal and the Defender portal's Threat Intelligence pages.

Malware Analysis Using Security Copilot: Copilot can explain a file's behavior, reputation, and relationships to known malware families when given a hash, filename, or incident reference — grounded in your tenant's actual detonation/detection data plus global threat intelligence.

Threat Intelligence Enrichment — key promptbooks:

Promptbook	Purpose
Threat actor profile	Report profiling a known threat actor and defenses against their tools/tactics
Vulnerability impact assessment	Summarizes a known CVE and remediation steps
Check impact of an external threat article	Extracts IOCs from a pasted article and generates hunting queries
Threat Intelligence 360 report	Same, but sourced from a Defender TI article specifically

Hands-on Lab 12.5 — Enrich Your Lab's Malware Alert

1. In Security Copilot standalone, open **Sources** → confirm the **Microsoft Threat Intelligence** plugin is turned on.
2. Prompt: `Summarize threat intelligence data for Trojan:Win32/Vigorf.A` (the detection name from Part 10.5's demonstration scenario, or whatever malware family your simulation triggered).
3. Run the **Threat actor profile** promptbook if your simulation's alert names a specific tracked actor/campaign; otherwise, prompt: `What tools and techniques are commonly associated with this malware family, and how would I detect them in Sentinel?`
4. Back in the Defender portal, open your Part 10.5 malware-related alert → use the embedded Copilot pane's suggested prompts to pull related file reputation and IOC data directly into the incident context.
5. Cross-reference: prompt Copilot to `Generate a KQL query to hunt for any other devices that have seen file hash <SHA256 from the alert>` — run it against your workspace.

12.6 Incident Response Automation

AI-Assisted Incident Response: Beyond summaries, Copilot offers **guided responses** — specific, incident-tailored remediation suggestions (isolate device, block sender, reset credentials) surfaced directly on the incident page.

Security Copilot Plugins: Three types exist — **API plugins** (call an external REST API), **GPT plugins** (add domain knowledge), and **KQL plugins** (run a parameterized KQL query against Sentinel, Defender, or an Azure Data Explorer/Kusto cluster and feed the results back into a Copilot session). You already have everything needed to build a custom KQL plugin against your own lab table.

SOAR Integration: Security Copilot promptbooks can be triggered from **Azure Logic Apps**, letting you chain a Sentinel automation rule → Logic App → Copilot promptbook, so

incident enrichment happens automatically the moment a new incident is created — no analyst has to remember to run it.

Hands-on Lab 12.6 — Build a Custom KQL Plugin Against `SentinelLab_CL`

1. On a machine with VS Code (recommended, to avoid YAML indentation errors), create a file `SentinelLabPlugin.yaml`:

```
yaml

Descriptor:
  Name: SentinelLabInsiderThreat
  DisplayName: Sentinel Lab - Insider Threat Lookup
  Description: Skills to query the lab's insider threat / data exfiltration eve
SkillGroups:
  - Format: KQL
    Skills:
      - Name: GetHighVolumeDataMovers
        DisplayName: Get Users With High Data Movement
        Description: Finds users who moved more than a specified volume of data
        Inputs:
          - Name: minMB
            Description: Minimum total MB moved to flag a user
            Required: true
        Settings:
          Target: Sentinel
          TenantId: '{{TenantId}}'
          SubscriptionId: '{{SubscriptionId}}'
          ResourceGroupName: 'rg-sentinel-lab'
          WorkspaceName: 'law-sentinel-lab'
          Template: |-
            SentinelLab_CL
            | where EventType in ("FileAccess", "CloudUpload")
            | summarize TotalMB = sum(DataVolumeMB) by Username
            | where TotalMB > {{minMB}}
            | sort by TotalMB desc
```

2. Fill in your actual `TenantId` and `SubscriptionId` values (from earlier parts of this lab).
3. In Security Copilot, go to **Sources** → **Plugins** → + **Add plugin** (or **Create plugin**, naming varies by portal version) → upload `SentinelLabPlugin.yaml`.
4. Turn the new plugin on, then prompt: `Get users with high data movement, minimum 100 MB.` — Copilot should invoke your custom skill directly rather than trying to write its own KQL.
5. Discuss with the class: this pattern is how a real SOC bakes its own tuned/vetted detection queries into Copilot, instead of relying on the AI to reconstruct them from

scratch each time.

✓ **Also upload the companion brute-force plugin before continuing to 12.9:** the `SentinelLabBruteForce` plugin (four skills — burst detection, per-user failure lookup, success-after-failure correlation, and IP timeline) follows the exact same pattern as above and is required for the full investigation promptbook you'll build in 12.9. Upload it the same way: **Sources → Plugins → Add plugin → Upload file**, filling in your `TenantId`/`SubscriptionId` as you did for this one.

SOAR automation discussion (no separate lab required): Walk through the pattern conceptually — a Sentinel automation rule triggers on a new incident, calls a Logic App, and the Logic App invokes a Copilot promptbook via API, writing the resulting summary back as an incident comment. Reference **Sentinel → Automation → Playbooks** as the place this would be built in a production tenant.

12.7 Advanced SOC Use Cases

Run each of these against the synthetic data you already ingested in Part 6 — this reinforces that Copilot's value is in accelerating investigation of data you understand, not just novel scenarios.

Identity Attack Investigations:

Summarize sign-in activity for user mgarcia@contoso.com over the last 7 days, and flag anything unusual.

(Should surface the impossible-travel pattern from Part 6.6.)

Cloud Security Investigations:

Show me any cloud upload activity to external domains in the last 24 hours, and assess the risk.

(Should surface the Part 6.4 CloudUpload/personal-dropbox pattern.)

Insider Threat Detection:

Which users have downloaded or uploaded unusually large volumes of data in the last week, and what files were involved?

Hands-on Lab 12.7 — Run All Three and Compare to Your Hand-Built Rules

1. Run each prompt above against your workspace (via the Sentinel plugin from 12.4).
2. For each, compare Copilot's finding to the Part 8 Analytics Rule you built for the same scenario — did it find the same entities? Same threshold logic?

3. Discussion: where did Copilot's answer differ from your rule, and why? (Common answer: Copilot infers a reasonable threshold from context rather than using your exact hardcoded value — a good teaching moment on why hand-tuned detection rules still matter alongside AI assistance.)
-

12.8 SOC Reporting and Executive Communication

Executive Reporting: The **Report** button on any incident (12.3, step 5) generates a plain-language summary suitable for non-technical stakeholders — timeline, business impact framing, and recommended actions, without raw log data.

Compliance Reporting: Prompt Copilot to frame findings against a specific framework, e.g.

Summarize this incident in terms relevant to a SOC 2 audit or List which NIST CSF functions this incident touches. Always have a human compliance owner review AI-generated compliance framing before it's used externally — Copilot output is a drafting aid, not an attestation.

Security Metrics and Dashboards: The **usage monitoring dashboard** (Security Copilot → **Home** → **Usage monitoring**) shows SCU consumption by plugin, session, and user — useful both for cost management and as a proxy metric for adoption across the SOC team.

Hands-on Lab 12.8 — Build an Executive Summary Package

1. Take the incident **Report** generated in 12.3 and the malware enrichment output from 12.5.
 2. Prompt: Combine the findings from the brute-force incident and the malware detection into a single executive summary suitable for a board-level security update, under 300 words.
 3. Go to **Home** → **Usage monitoring** → review your class's SCU consumption so far — filter by plugin to see which activities (KQL generation vs. incident summarization vs. plugin calls) consumed the most capacity.
 4. Discussion: have students draft one compliance-framed sentence for their incident, and compare phrasing across the class — reinforces that the human review step matters even with good AI drafting.
-

12.9 Advanced Security Copilot Operations

Custom Promptbooks: A promptbook is a saved sequence of prompts (optionally parameterized) that run as one repeatable workflow — the SOC-authored equivalent of the built-in "Incident investigation" promptbook you used in 12.4.

Copilot Extensibility: Beyond KQL plugins (12.6), Security Copilot supports **API plugins** (OpenAPI-described REST calls), **GPT plugins**, and — per the newest capability — **Agent**

Builder, letting teams create custom autonomous agents, plus API/MCP-based integration for programmatic control.

Security Copilot Governance: Role model to teach:

Role	Capability
Owner	Manage capacity, plugins, roles, billing
Contributor	Use Copilot, manage some settings, can't touch billing
Everyone (default group)	Broadest — replace with security-scoped groups in production, never leave as default in a real tenant

Hands-on Lab 12.9 — Build a Custom Promptbook: Full Brute Force Investigation

i Promptbooks aren't uploaded as a YAML file the way plugins (12.6) and agents (12.11) are — they're built directly in the portal, either from an existing chat session's prompts or step-by-step in the promptbook builder. The lab below uses the builder directly so every step is explicit and reusable by the whole class.

This promptbook chains **seven** prompts into one repeatable investigation, pulling from both custom plugins you've already built: `SentinelLabBruteForce` (the four-skill brute-force plugin) and `SentinelLabInsiderThreat` (the data-exfiltration plugin from 12.6). Each step builds on the context Copilot already gathered in the steps before it.

1. Prerequisite: confirm both plugins are turned on under **Sources → Plugins** before building the promptbook, or its steps will fail to resolve when run.

2. Create the promptbook:

1. Security Copilot → **Promptbooks** (left nav) → **Create promptbook**.
2. **Name:** `Lab - Brute Force Full Investigation`
3. **Description:** `Investigates a brute-force login pattern end to end: detects the burst, confirms whether it led to a successful takeover, checks the affected user for follow-on data exfiltration and sign-in anomalies, and produces an executive summary.`
4. **Tags:** `brute-force`, `credential-stuffing`, `lab`

3. Add the seven prompt steps in order — use angle-bracket placeholders exactly as shown; the promptbook builder turns these into input fields automatically when you save:

Step Prompt

- 1 `Get brute force source IPs with minimum <MIN_FAILURES> failures in a <WINDOW_MINUTES> minute window.`
- 2 `For source IP <SOURCE_IP>, get the full brute force timeline.`
- 3 `Check for success after brute force for source IP <SOURCE_IP> within <WITHIN_MINUTES> minutes.`
- 4 `Get failed logins for user <USER_NAME> in the last <LOOKBACK_HOURS> hours.`
- 5 `Get users with high data movement, minimum <MIN_MB> MB.`
- 6 `Summarize sign-in activity for <USER_NAME> over the last 7 days, and flag anything unusual.`
- 7 `Combine the findings from all steps above into an executive summary under 200 words. Explicitly flag this as "High Priority - Possible Account Takeover with Data Exfiltration" if step 3 shows a successful login after the brute-force burst AND step 5 shows the same user among the high-volume data movers.`

Step 1 and step 5 route to your `SentinelLabBruteForce` and `SentinelLabInsiderThreat` plugins respectively; steps 4, 6 use the built-in Sentinel/Defender natural-language capabilities from 12.4/12.7; step 7 is pure synthesis with no data source, relying entirely on the prior steps' context.

4. Configure sharing: at the bottom of the builder, set visibility to **Anyone in my organization** (so the whole class can reuse it) or **Just me**, depending on your workspace policy → **Create**.

5. Run it end to end: open the saved promptbook → **Run** → supply sample values:

```
MIN_FAILURES = 10
WINDOW_MINUTES = 10
SOURCE_IP = 203.0.113.45
WITHIN_MINUTES = 15
USER_NAME = jdoe@contoso.com
LOOKBACK_HOURS = 24
MIN_MB = 100
```

This should walk straight through your Part 6.1 synthetic brute-force scenario, correctly flag it as high priority in step 7's summary (since 6.1's data includes both the success-after-failures pattern and, if you also loaded 6.4's data for the same tenant, matching exfiltration volume), and produce a clean executive summary.

6. Governance exercise: go to **Settings** → **Roles**, review who currently has Owner/Contributor, and have students propose (on paper/discussion, not actually applied) a least-privilege role assignment for a fictional 5-person SOC team — tier-1 analysts, tier-2, SOC manager, and an auditor. Discuss specifically: should tier-1 analysts be able to **edit** this promptbook, or only **run** it?

12.10 SOC Capstone Project

Assign this as a graded/reviewed exercise combining everything from Parts 1-12. Students should work in the same lab environment, using their own ingested data.

Scenario A — Ransomware Investigation: Using the Part 10 Controlled Folder Access / ransomware demonstration scenario (or the Part 6.3/6.4-style synthetic mass-file-rename data), have students:

1. Investigate the resulting incident/alert using Copilot in Defender.
2. Generate a KQL hunting query (via Copilot) to find any other devices exhibiting the same file-rename pattern.
3. Produce an executive report and a compliance-framed summary.

Scenario B — Phishing Attack Investigation: Using the Part 6.5 synthetic `EmailEvents`-style phishing data (or real Defender for Office 365 data if licensed):

1. Prompt Copilot to identify all recipients of the phishing campaign and any who clicked/interacted.
2. Use the Threat Intelligence promptbooks (12.5) to enrich the sender domain.
3. Build a custom KQL plugin skill (12.6 pattern) specifically for "find all emails from a given sender domain," and demonstrate it live.

Scenario C — Insider Threat Investigation: Using the Part 6.4 data exfiltration dataset:

1. Use the Identity + Cloud prompts from 12.7 to build the full picture of the user's activity.
2. Draft a custom promptbook (12.9 pattern) that automates this specific investigation end to end.

Final Deliverables (each student/team submits):

1. One incident **Report** (12.3/12.8 style) for their assigned scenario.
2. One custom KQL plugin (12.6) tailored to their scenario.
3. One custom promptbook (12.9) automating their scenario's investigation.
4. A one-page executive summary (12.8) suitable for a non-technical stakeholder.
5. A short (5-minute) live demo running their promptbook against the lab environment.

Grading rubric suggestion:

Criterion	Weight
Correctness of KQL plugin logic	25%
Promptbook design (clarity, parameterization, reuse of earlier lab work)	25%
Quality/accuracy of executive summary	25%
Live demo execution	25%

12.11 Security Copilot Agent Development (Deep Dive)

This expands on the brief mention of Agent Builder in 12.9 into a full build-test-publish workflow — the same pattern used for both Microsoft-built and custom agents.

What an agent actually is: unlike a promptbook (a fixed sequence of prompts), an agent reasons dynamically — it plans its own steps, decides which tools to call and in what order, and can react to triggers rather than only running on demand. Every agent is built from five components:

Component	Role
Tools (Skills)	The functions/actions the agent can call — KQL, API, GPT, MCP, or another agent (AGENT type)
Triggers	What starts a run — manual invocation, a schedule, or an event (e.g. new incident created)
Orchestrator	The reasoning logic that decides which tools to call and in what order to meet the goal
Instructions	Natural-language system directives constraining how the agent behaves
Feedback	Prior run outcomes stored and used to inform future runs

Three developer personas: Developers (build agents, possibly for cross-org distribution via Security Store), Administrators (decide which agents get installed org-wide, review usage metrics), and end users (invoke published agents). RBAC in Security Copilot is separate from Entra/Azure RBAC — a user's ability to **build**, **test**, and **publish** (at user scope vs. workspace scope) is governed by Security Copilot roles specifically.

Two ways to build, same output: whether you use the no-code Build page or hand-write YAML, both converge on the same underlying **agent manifest** (a YAML file) that gets deployed to Security Copilot. Pick whichever fits the person building it — the lab below covers both.

Hands-on Lab 12.11a — Build a Brute Force Response Agent (No-Code, Portal Click-by-Click)

We'll build `LabBruteForceResponseAgent`, wiring together all five skills you now have across both custom plugins: the four `SentinelLabBruteForce` skills (12.6 companion upload) plus `GetHighVolumeDataMovers` from `SentinelLabInsiderThreat` (12.6). Unlike the 12.9 promptbook — which runs a **fixed** sequence of steps every time — this agent decides for itself, at run time, which skills to call and in what order based on what it finds.

Prerequisite: confirm both `SentinelLabBruteForce` and `SentinelLabInsiderThreat` are turned on under **Sources** → **Plugins** before starting.

1. Start the build:

1. Security Copilot → **Build** (left nav) → **Start from scratch** → **Get started**.
2. Note the two tabs: **Build** (define the agent) and **Test** (validate responses), and the **Agent overview** pane listing: **Details, Instructions, Inputs, Tools, Automation**.

2. Details:

- **Agent display name:** `LabBruteForceResponseAgent` (no spaces — this is how you'll search for it later)
- **Agent description:** Investigates a brute-force login pattern end to end: detects the burst, checks whether it escalated to a successful login, and checks the affected user for follow-on data exfiltration.

3. Instructions — this is what lets the agent reason dynamically instead of following a fixed script, so be explicit about the decision logic, not just the goal:

You investigate brute-force login activity in SentinelLab_CL.

Given a SourceIP, minFailures, and windowMinutes:

1. Use GetBruteForceSourceIPs to confirm this IP shows a burst matching or exceeding minFailures within windowMinutes. If it does not, report "No brute-force pattern confirmed for this IP" and stop.
2. If confirmed, use GetSuccessAfterBruteForce to check whether any of the affected usernames had a successful login shortly after the burst.
3. For each username flagged as a successful takeover in step 2, use GetHighVolumeDataMovers with minMB=100 to check whether that same user also moved a large volume of data.
4. Use GetBruteForceTimeline to pull the full event sequence for the IP as supporting evidence.

Summarize your findings in under 150 words. Explicitly flag the result as "High Priority - Possible Account Takeover with Data Exfiltration" only if BOTH a successful takeover (step 2) AND high-volume data movement (step 3) are confirmed for the same user. Otherwise, state clearly which conditions were and were not met.

4. Inputs: add three input parameters:

Name	Description	Required
sourceIP	The source IP address to investigate	true
minFailures	Minimum failed logins to count as a burst	false (agent should default to 10 per its instructions)
windowMinutes	Time window in minutes for the burst check	false (default 10)

5. Tools: select **Add tool** → **Tools catalog** → add all four SentinelLabBruteForce skills (GetBruteForceSourceIPs, GetBruteForceTimeline, GetSuccessAfterBruteForce, GetFailedLoginsByUser) plus GetHighVolumeDataMovers from SentinelLabInsiderThreat — five tools total. Unlike the promptbook, you're not specifying the call order here; the orchestrator decides that at run time based on your Instructions.

6. Alternative build method: select the Copilot chat button on the right and describe the agent conversationally instead — paste the Instructions text from step 3 and Copilot will scaffold Details/Inputs/Tools for you. You can freely switch between chat and the form to refine either way.

7. Scoping: set Single tenant constraint to Workspace (one instance per workspace — right for this lab; use Tenant for an org-wide singleton, or None to allow multiple parallel

instances).

8. Test it:

1. Switch to the **Test** tab → run with `sourceIP = 203.0.113.45`, `minFailures = 10`, `windowMinutes = 10` (the Part 6.1 attacker).
2. Review the **step-by-step activity map** — confirm the agent called the tools in the order your Instructions implied (burst check → success check → exfil check → timeline), not just all five regardless of relevance.
3. Check the **Activity graph** tab for the visual node map; click any node for that call's exact input/output.
4. Now test the **negative case**: run again with a made-up IP like `198.51.100.200` that has no data at all. Confirm the agent correctly reports "No brute-force pattern confirmed" and does **not** call the exfiltration/timeline tools unnecessarily — a well-instructed agent should short-circuit here rather than burning SCUs on irrelevant calls.

9. Turn on Autotune instruction optimization and re-run both tests. Review the AI quality scoring (clarity, grounding, detail) and any recommended instruction rewording — copy improvements back into step 3 if they sharpen the negative-case behavior in particular, since that's the case most agents get wrong first.

10. Confirm the YAML view stays in sync with your form edits (useful handoff artifact for 12.11b/12.11c).

11. Publish: choose **Workspace** scope → confirm.

12. Invoke it: prompt bar → **Prompts** icon → **See all system capabilities** → search `LabBruteForceResponseAgent` → supply the three inputs → **Submit**. Expect ~30-45 seconds. Select **Agent view** on the response to verify which tools it actually called for this run.

13. Compare to the promptbook (12.9): as a class discussion — the promptbook always runs all 7 steps in the same order regardless of what it finds; the agent skipped straight to "no pattern confirmed" on the negative test. Ask students which behavior they'd want for a scheduled/automated trigger versus an analyst-initiated investigation, and why.

12.11b — The YAML Manifest (Pro-Code Path)

For teams that prefer to build/maintain agents as code, the same agent can be authored directly as YAML and uploaded rather than using the Build page. This version references the skills from your **already-published** `SentinelLabBruteForce` and `SentinelLabInsiderThreat` plugins directly, rather than redefining any KQL inline — the cleaner approach once those plugins exist, since it avoids maintaining the same query logic in two places:

```
yaml
```

Descriptor:

Name: LabBruteForceResponseAgent

DisplayName: Lab Brute Force Response Agent

Description: >

Investigates a brute-force login pattern end to end: detects the burst, checks whether it escalated to a successful login, and checks the affected user for follow-on data exfiltration.

AgentDefinitions:

- **Name:** LabBruteForceResponseAgent

Instructions: >

You investigate brute-force login activity in SentinelLab_CL.

Given a SourceIP, minFailures, and windowMinutes:

1. Use GetBruteForceSourceIPs to confirm this IP shows a burst matching or exceeding minFailures within windowMinutes. If it does not, report "No brute-force pattern confirmed for this IP" and stop.
2. If confirmed, use GetSuccessAfterBruteForce to check whether any affected username had a successful login shortly after the burst.
3. For each username flagged as a takeover in step 2, use GetHighVolumeDataMovers with minMB=100 to check for high-volume data movement by that same user.
4. Use GetBruteForceTimeline to pull the full event sequence for the IP as supporting evidence.

Summarize findings in under 150 words. Flag "High Priority - Possible Account Takeover with Data Exfiltration" only if BOTH a successful takeover AND high-volume data movement are confirmed for the same user; otherwise state clearly which conditions were and were not met.

Inputs:

- **Name:** sourceIP

Description: The source IP address to investigate

Required: true

- **Name:** minFailures

Description: Minimum failed logins to count as a burst

Required: false

- **Name:** windowMinutes

Description: Time window in minutes for the burst check

Required: false

Tools:

- **Name:** SentinelLabBruteForce.GetBruteForceSourceIPs

- **Name:** SentinelLabBruteForce.GetBruteForceTimeline

- **Name:** SentinelLabBruteForce.GetSuccessAfterBruteForce

- **Name:** SentinelLabBruteForce.GetFailedLoginsByUser

- **Name:** SentinelLabInsiderThreat.GetHighVolumeDataMovers

Key points on the manifest:

- `Descriptor` names/describes the agent package as a whole.
- `AgentDefinitions` is what makes this an **agent** rather than a plain plugin — a YAML file is only published as an agent (visible in **Active agents**) if it contains an `AgentDefinitions` block. This manifest has **no** `SkillGroups` **block at all**, because it doesn't need to define any new KQL — every tool it references already exists in the two plugins you published in 12.6.
- `Tools` entries use the `<PluginName>.<SkillName>` format to reference skills from other already-published plugins — this is why both `SentinelLabBruteForce` and `SentinelLabInsiderThreat` must be uploaded and turned on **before** you upload this agent manifest, or the tool references will fail to resolve.
- Upload via **Sources** → **Add plugin** → **Upload file** (or **Upload as a link** for a hosted YAML/JSON). Set visibility to **just yourself** (private tag) or **anyone in the organization** — the latter requires an Owner to have enabled org-wide custom plugin management first.
- Discovery works the same as the no-code path: search by `DisplayName` under **See all system capabilities**.

12.11c — Optional Advanced Track: Building in VS Code with the Sentinel MCP Server

For developer-heavy classes, mention this as the professional path without necessarily doing it live:

1. Enable the **Microsoft Sentinel MCP server** in VS Code — this exposes Security Copilot agent-creation MCP tools directly to GitHub Copilot inside the editor.
2. Describe the desired agent in natural language to GitHub Copilot Chat, grounded in real Sentinel data/schema context via the MCP connection.
3. GitHub Copilot scaffolds the YAML manifest for you, informed by your actual workspace's tables and columns (so it won't hallucinate a column name that doesn't exist in `SentinelLab_CL`, for example).
4. Iterate and test using the same MCP tools, then publish to Security Copilot exactly as in 12.11b.

Discussion point for the class: this pro-code path is the same manifest format as the no-code Build page — there's no lock-in either direction. A reasonable team workflow is: analysts prototype quickly with the Build page's chat interface, then developers take the generated YAML into source control for versioning, code review, and CI/CD once an agent is ready to promote beyond one person's private use.

12.11d — Governance for Custom Agents (discussion + checklist)

Before publishing any custom agent beyond personal/private scope, walk the class through a review checklist — this matters more for agents than promptbooks, since agents can chain multiple tool calls autonomously:

- ☐ Does the agent's **Instructions** field clearly bound its scope (what it should and shouldn't do)?
- ☐ Are all **Tools** it can call read-only, or does any tool take an action (isolate device, block sender, delete file)? Action-taking tools need explicit human-approval gates — check the **Automation → Permissions** section for this agent.
- ☐ Is the **Single tenant constraint** set correctly (Workspace scope prevents accidental tenant-wide duplication)?
- ☐ Has it been tested with **autotune** and reviewed for grounding quality, not just "does it run without erroring"?
- ☐ Who is the accountable **Owner** if this agent produces an incorrect verdict in production?

This maps directly back to the Governance role table from Lab 12.9 — Owners and Contributors are the ones who can promote an agent from private/tested to workspace-wide availability, so the review step belongs with them, not with whoever happened to build the agent first.

Part 13 — Cleanup (avoid ongoing charges)

1. Portal → **Resource groups** → `rg-sentinel-lab` → **Delete resource group** → type the name to confirm.
 - This removes the workspace, Sentinel, the DCR (`dcr-sentinel-lab`), and its DCE together.
 2. Portal → **App registrations** → `sentinel-lab-ingest-app` → **Delete**.
 3. If you connected a real Defender XDR trial tenant in Part 10, also complete the offboarding and connector-disconnect steps in **10.10** — deleting the Azure resource group alone does not touch the M365 tenant.
 4. If you provisioned standalone Security Copilot capacity in Part 12.1, delete it separately: Security Copilot → **Settings** → **Owner settings** → **Security compute units** → **Change** → reduce to 0 / delete capacity. This is billed independently of everything else in this guide and won't be removed by deleting `rg-sentinel-lab` unless the SCU capacity resource itself lives in that resource group.
-

Appendix A — Splitting Into Separate Tables Per Use Case (optional variant)

If you'd rather have `BruteForce_CL`, `Malware_CL`, `NetworkAnomaly_CL`, etc. as separate tables (cleaner for teaching schema design), repeat Part 4.1-4.3 once per table, giving each its own DCR/stream name, and point each PowerShell script at its own `$Cfg`. The ingestion function in Part 5 doesn't change — only the `$Cfg` values per script.

Appendix B — Cost Notes for Instructors

- Log Analytics charges per GB ingested/retained. Synthetic lab data volumes above are trivial (a few KB per run).
 - Set workspace **Daily cap** (Workspace → **Usage and estimated costs** → **Data Cap**) to something small (e.g. 1 GB/day) to guard against runaway ingestion during a class.
 - Sentinel itself bills based on GB ingested into the workspace — same trivial cost for lab-scale data.
-

Appendix C — Troubleshooting Guide (real issues encountered building this lab)

These are the actual failure modes hit while validating this guide end-to-end, in the order you're likely to encounter them, with root cause and fix. Worth reading *before* you start, not just when something breaks.

C.1 — 400 Bad Request, generic message, no detail

Symptom: `Ingestion failed: The remote server returned an error: (400) Bad Request.`

Cause: `Invoke-RestMethod`'s default exception (`($_.Exception.Message)`) only shows the HTTP status wrapper — it hides Azure's actual JSON error body, which is the only thing that explains *why*. **Fix:** Use the hardened catch block from Part 5 (already updated above), which reads the response stream directly and prints the real `BODY:`. Never troubleshoot ingestion errors without this — every other issue below was only diagnosable once this was in place.

C.2 — 403 `OperationFailed`: "authentication token provided does not have access"

Symptom: Token is valid (decodes fine, right `(appid)/(tid)/(aud)`), role assignment exists and looks correct in `(Get-AzRoleAssignment)`, but still 403. **Cause:** Azure Monitor's data-plane authorization cache for a newly created DCR role assignment can take **up to 20-30 minutes** to propagate, even though the control-plane (`(Get-AzRoleAssignment)`) shows it as active immediately. This is the single most time-consuming issue in this lab, purely because it's easy to retest too early and conclude something else is wrong. **Fix:** Confirm role scope is exactly the DCR resource, confirm identity (`(appid)` in token) matches the service principal the role was granted to, then **wait a genuine, uninterrupted 15-20 minutes** with no retries, get a **fresh token** (don't reuse an old one), and retest. Removing and re-adding the role assignment can reset a stuck propagation state faster than waiting on the original one.

Ruled out along the way, in case you hit similar symptoms: subscription-level Owner role (doesn't grant Monitor data-plane `(DataActions)`, only management-plane `(Actions)`), DCE network restrictions (check `(Get-AzDataCollectionEndpoint ... | Format-List *)` for `(NetworkAclsPublicNetworkAccess)`), Azure Policy deny effects (`(Get-AzPolicyAssignment)`), and Blueprint-style deny assignments (`(Get-AzDenyAssignment)`).

C.3 — 400 `InvalidRequestPath`: "The request path is invalid"

Symptom: Everything (auth, RBAC, DCR, DCE) has been verified correct,

`$Cfg.StreamName` prints correctly on its own, but the ingestion call still 400s. **Cause:** A

hidden/control character (typically a stray carriage return `\r`) from copying a value out of the Azure Portal) embedded inside one of the config strings. String interpolation

`("$DceUri/.../$StreamName")` silently tolerates this and can produce a URL that displays as correct in some contexts but is actually truncated or malformed when sent over the wire.

Fix: Rebuild the URI with explicit `.Trim()` on every component (see the hardened function in Part 5), and add a temporary `Write-Host "DEBUG URI: [$uri]"` with square brackets around it — brackets make truncation or stray whitespace immediately visible. Also useful: check for hidden characters directly with `[System.Text.Encoding]::UTF8.GetBytes($value) | ForEach-Object { $_.ToString("X2") }` and look for `0D` (CR) or `0A` (LF) bytes.

C.4 — Table name doubled: `SentinelLab_CL_CL`

Symptom: Everything else works, but the "correct" stream name from the guide (`Custom-SentinelLab_CL`) 400s, while `Custom-SentinelLab_CL_CL` succeeds. **Cause:** Azure

automatically appends `_CL` to custom log table names. Typing `SentinelLab_CL` into the table-name field produces `SentinelLab_CL_CL` — the suffix gets doubled. **Fix:** Type only

`SentinelLab` (no suffix) when creating the table (see Part 4.1's naming pitfall note). If a

table was already created with the doubled name, either use `SentinelLab_CL_CL` consistently everywhere (works fine, just needs consistency across `$Cfg`, KQL queries, and Analytics Rules), or delete and recreate the table with the correct name. **Always verify, don't assume:** Log Analytics workspace → **Tables** shows the real name. DCR → **JSON view** → `dataFlows[0].streams` shows the real stream name. Use those, not what the guide (or your memory) says it "should" be.

C.5 — `'summarize'` / `'where' operator: Failed to resolve scalar expression named 'X'`

Symptom: Ingestion reports success, but a KQL query referencing a specific column (e.g. `DestinationPort`, `Action`) fails to resolve it — even though the field was clearly present in the PowerShell object that was sent. **Cause:** DCR-based custom tables only persist columns

that exist in the table's declared schema. Any field in the ingested JSON that isn't a known column is **silently dropped** — the API returns success because ingestion of the *known* fields succeeded, not because every field was stored. **Fix:**

- Preventively: create the table with a combined sample covering every use case's fields (Part 4.1, step 6) so the full schema exists from the start.
- Reactively: check `<table> | getschema` to see exactly what's stored, then add missing columns via **Tables** → **Edit schema**, or push the full column set in one atomic call via the ARM REST API (`Invoke-AzRestMethod ... -Method PATCH`) rather than adding them one at a time through the portal UI — the one-at-a-time flow is easy to lose a column from mid-session.
- After adding columns, previously-ingested records that were missing those fields **do not get backfilled** — you must resend that use case's data.

C.6 — General debugging approach that worked

When something fails and the cause isn't obvious:

1. Get the real error body first (C.1) — never guess from the generic exception message.
2. Isolate with a single-record test script separate from the full use-case scripts, so you're not debugging "is it the payload volume or something else."
3. Verify each config value against its actual source in the portal, not against what a script or guide assumes it should be.
4. Change one variable at a time (e.g. don't fix RBAC and schema simultaneously) so you know which fix actually resolved which symptom.